

Solving the Space Optimization Competition 4 Challenge 2 of the European Space Agency

— Thomas Weise, Sina Abdipoor, Zhize Wu, and Mingxuan Li —

Thomas Weise (汤卫思)

tweise@hfu.edu.cn · <https://thomasweise.github.io>

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Introduction



- The Space Optimization Competition (SpOC) is organized by the European Space Agency (ESA) at the Genetic and Evolutionary Computation Conference (GECCO).

Thomas Weise (汤卫恩), Sina Abdipoor (思纳), Mingxuan Li (李明轩), and Zhize Wu (吴志泽). "Solving the SpOC 4 Challenge 2 with a Multi-Step Permutation Encoding for the Keplerian Traveling Salesperson". In: *28th Genetic and Evolutionary Computation Conference Companion (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. Association for Computing Machinery (ACM), 2026. ISBN: 979-8-4007-2488-6. doi:10.1145/3795101.3815557

Introduction



- The Space Optimization Competition (SpOC) is organized by the European Space Agency (ESA) at the Genetic and Evolutionary Computation Conference (GECCO).
- It poses optimization problems and challenges from space exploration and travel.

Thomas Weise (汤卫恩), Sina Abdipoor (思纳), Mingxuan Li (李明轩), and Zhize Wu (吴志泽). "Solving the SpOC 4 Challenge 2 with a Multi-Step Permutation Encoding for the Keplerian Traveling Salesperson". In: *28th Genetic and Evolutionary Computation Conference Companion (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. Association for Computing Machinery (ACM), 2026. ISBN: 979-8-4007-2488-6. doi:10.1145/3795101.3815557

Introduction



- The Space Optimization Competition (SpOC) is organized by the European Space Agency (ESA) at the Genetic and Evolutionary Computation Conference (GECCO).
- It poses optimization problems and challenges from space exploration and travel.
- We are the **ScholORs_HFUU+Sunway** team, a cooperation between researchers from Hefei University (合肥大学), China and Sunway University, Malaysia.

Thomas Weise (汤卫思), Sina Abdipoor (思纳), Mingxuan Li (李明轩), and Zhize Wu (吴志泽). "Solving the SpOC 4 Challenge 2 with a Multi-Step Permutation Encoding for the Keplerian Traveling Salesperson". In: *28th Genetic and Evolutionary Computation Conference Companion (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. Association for Computing Machinery (ACM), 2026. ISBN: 979-8-4007-2488-6. doi:10.1145/3795101.3815557

Introduction



- The Space Optimization Competition (SpOC) is organized by the European Space Agency (ESA) at the Genetic and Evolutionary Computation Conference (GECCO).
- It poses optimization problems and challenges from space exploration and travel.

Graph Theory
SpOC 4: Keplerian Tomato Traveling Salesperson / Small

Beginner 1 objective 145 dimensions

Publication Date
20 March 2026

Description
Small number of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: small

Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-small-1775056696477.py>

Top score
138.753
[SchoIRs_HFUI](#)

Leaderboard

USERNAME	SCORE	SUBMISSION DATE
SchoIRs_HFUI+Sunway	138.753	Apr 7, 2026, 7:49 AM

U+Sunway team, a cooperation between researchers from
, China and Sunway University, Malaysia.

This work was presented by Mingxuan Li (李明轩), and Zhize Wu (吴志泽). "Solving the SpOC 4 Challenge 2 with a Multi-Step Permutation Encoding for the Keplerian Traveling Salesperson". In: *28th Genetic and Evolutionary Computation Conference Companion (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. Association for Computing Machinery (ACM), 2026. ISBN: 979-8-4007-2488-6. doi:10.1145/3795101.3815557

Introduction



- The Space Optimization Competition (SpOC) is organized by the European Space Agency (ESA) at the Genetic and Evolutionary Computation Conference (GECCO).
- It poses optimization problems and challenges from space exploration and travel.

U+Sunway team, a cooperation between researchers from University, Malaysia.

Graph Theory
SpOC 4: Keplerian Tomato Tr Salesperson / Small

Beginner 1 objective

Publication Date
20 March 2026

Description
Small number of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: small
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-small-1775056696477.py>

Leaderboard

USERNAME	SCORE
SchoIRs_HFUU+Sunway	138.753

Graph Theory
SpOC 4: Keplerian Tomato Traveling Salesperson / Medium

Intermediate 1 objective 541 dimensions

Publication Date
23 March 2026

Description
A very average amount of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: medium
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-medium-177505662015.py>

Leaderboard

USERNAME	SCORE	SUBMISSION DATE
SchoIRs_HFUU+Sunway	463.611	Apr 8, 2026, 4:10 PM

Thomas B. Schone, "Solving the SpOC 4 Challenge 2 with a Multi-Step Permutation Encoding for the Keplerian Tomato Traveling Salesperson Companion (GECCO'2026). July 1-5, 2026, Association for Computing Machinery (ACM), 2026. ISBN: 9781450399999

Vu (吴志泽). "Solving the SpOC 4 Challenge 2 with a Multi-Step and Evolutionary Computation Conference". In: Proceedings of the Genetic and Evolutionary Computation Conference. Trujillo and Ting Hu. Association for Computing Machinery, 2026. ISBN: 9781450399999

Introduction



- The Space Optimization Competition (SpOC) is organized by the European Space Agency (ESA) at the Genetic and Evolutionary Computation Conference (GECCO).
- It poses optimization problems and challenges in space and travel.



Graph Theory
SpOC 4: Keplerian Tomato Traveling Salesperson / Small

Beginner ■ 1 objective

Publication Date
20 March 2026

Description
Small number of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: small
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-small-1775056696477.py>

Leaderboard

USERNAME	SCORE
SchoIRs_HFUU+Sunway	138.753



Graph Theory
SpOC 4: Keplerian Tomato Traveling Salesperson / Medium

Intermediate ■ 1 objective

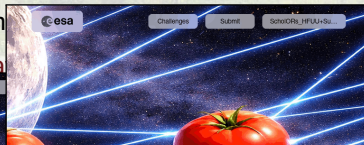
Publication Date
23 March 2026

Description
A very average amount of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: medium
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-medium-177505662015.py>

Leaderboard

USERNAME	SCORE	SUBMISSION DATE
SchoIRs_HFUU+Sunway	463.611	Apr 8, 2026, 4:10 PM



Graph Theory
SpOC 4: Keplerian Tomato Traveling Salesperson / Large

Advanced ■ 1 objective 3151 dimensions

Publication Date
23 March 2026

Description
A mildly oversized amount of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: large
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-large-1775056320972.py>

Top score
1,995.797
[SchoIRs_HFUU](#)

Leaderboard

USERNAME	SCORE	SUBMISSION DATE
SchoIRs_HFUU+Sunway	1,995.797	Apr 16, 2026, 5:47 AM
Team HFU	2,072.845	Apr 14, 2026, 4:04 PM

This book is a companion to the book "Permutation Encoding for the Keplerian Tomato Traveling Salesperson Problem" (GECCO'2026). July 1, 2026. ISBN: 978-1-60959-123-4

SpOC 4 Challenge 2 with a Multi-Step
Conference
association for Computing

Introduction



- The Space Optimization Competition (SpOC) is organized by the European Space Agency (ESA) at the Genetic and Evolutionary Computation Conference (GECCO).
- It poses optimization problems and challenges in space and travel.

Graph Theory

SpOC 4: Keplerian Tomato Traveling Salesperson / Small

Beginner 1 objective

Publication Date
20 March 2026

Description
Small number of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: small
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-small-1775056696477.py>

Leaderboard

USERNAME	SCORE
ScholORs_HFUU+Sunway	138.753

Graph Theory

SpOC 4: Keplerian Tomato Traveling Salesperson / Medium

Intermediate 1 objective

Publication Date
23 March 2026

Description
A very average amount of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: medium
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-medium-1775056662015.py>

Leaderboard

USERNAME	SCORE	SUBMISSION DATE
ScholORs_HFUU+Sunway	463.611	Apr 8, 2026, 4:10 PM

Graph Theory

SpOC 4: Keplerian Tomato Traveling Salesperson / Large

Advanced 1 objective

Publication Date
23 March 2026

Description
A mildly oversized amount of tomatoes
[Learn more](#)

Identifiers
challenge: spoc-4-keplerian-tomato-traveling-salesperson
problem: large
Evaluation code `</>`
<https://api.optimize.esa.int/media/problems/spoc-4-keplerian-tomato-traveling-salesperson-large-17750566320972.py>

Leaderboard

USERNAME	SCORE
ScholORs_HFUU+Sunway	1,995.797
Team HFE	2,072.845

China

ScholORs_HFUU+Sunway
Hefei University, China + Sunway University, Malaysia

[Edit profile](#)

Best submissions

PROBLEM	RANK	SCORE	SUBMISSION DATE
SpOC 4: Luna Tomato Advertising: Tie-breaker	1	37	Apr 16, 2026, 6:48 PM
SpOC 4: Keplerian Tomato Traveling Salesperson: Small	1	119.242	Apr 16, 2026, 12:16 PM
SpOC 4: Keplerian Tomato Traveling Salesperson: Medium	1	376.625	Apr 17, 2026, 5:59 AM
SpOC 4: Keplerian Tomato Traveling Salesperson: Large	1	1,995.797	Apr 16, 2026, 5:47 AM
SpOC 4: Luna Tomato Logistics: Matching II	4	-70,303.954	Apr 15, 2026, 7:36 PM
SpOC 4: Luna Tomato Logistics: Matching I	4	-32,894.923	Apr 15, 2026, 7:36 PM

Thomas Schödl, *Permutation Encoding for the Keplerian Tomato Traveling Salesperson Companion (GECCO'2026)*. July 1, 2026. ACM Press, New York, NY. ISBN: 978-1-4503-9781-2.

The Challenge



- Given are n objects orbiting our moon with their orbital data.

The Challenge



- Given are n objects orbiting our moon with their orbital data.
- There are three instances **small**, **medium**, and **large**, with $n = 49, 181, \text{ and } 1051$.

The Challenge



- Given are n objects orbiting our moon with their orbital data.
- There are three instances **small**, **medium**, and **large**, with $n = 49, 181, \text{ and } 1051$.
- The goal is to visit each of the objects exactly once as fast as possible, starting at an object of choice.

The Challenge



- Given are n objects orbiting our moon with their orbital data.
- There are three instances **small**, **medium**, and **large**, with $n = 49, 181, \text{ and } 1051$.
- The goal is to visit each of the objects exactly once as fast as possible, starting at an object of choice.
- For each transfer, our spacecraft may apply a specific change ΔV of velocity.

The Challenge



- Given are n objects orbiting our moon with their orbital data.
- There are three instances **small**, **medium**, and **large**, with $n = 49, 181, \text{ and } 1051$.
- The goal is to visit each of the objects exactly once as fast as possible, starting at an object of choice.
- For each transfer, our spacecraft may apply a specific change ΔV of velocity.
- ΔV is limited by ΔV_{max} , but 5 times we are allowed to do a larger velocity change $\Delta V \leq \Delta V_{max}^{exception}$.

The Challenge



- Given are n objects orbiting our moon with their orbital data.
- There are three instances **small**, **medium**, and **large**, with $n = 49, 181, \text{ and } 1051$.
- The goal is to visit each of the objects exactly once as fast as possible, starting at an object of choice.
- For each transfer, our spacecraft may apply a specific change ΔV of velocity.
- ΔV is limited by ΔV_{max} , but 5 times we are allowed to do a larger velocity change $\Delta V \leq \Delta V_{max}^{exception}$.
- Limits for the shortest transfer and maximum total time are also given.

Computing Transfers



- We begin by developing a tool $opt\Delta V$ that finds the earliest transfer from one object to another one within a given time window $[t_s, t_d]$, given a ΔV -limit ΔV_{max}^{lim} .

Computing Transfers



- We begin by developing a tool $opt\Delta V$ that finds the earliest transfer from one object to another one within a given time window $[t_s, t_d]$, given a ΔV -limit ΔV_{max}^{lim} .
- If no such transfer exists, the function returns an error.

Computing Transfers



- We begin by developing a tool $opt\Delta V$ that finds the earliest transfer from one object to another one within a given time window $[t_s, t_d]$, given a ΔV -limit ΔV_{max}^{lim} .
- If no such transfer exists, the function returns an error.
- We approach this as numerical minimization problem with objective function ψ :

Computing Transfers



- We begin by developing a tool $opt\Delta V$ that finds the earliest transfer from one object to another one within a given time window $[t_s, t_d]$, given a ΔV -limit ΔV_{max}^{lim} .
- If no such transfer exists, the function returns an error.
- We approach this as numerical minimization problem with objective function ψ :

$$\psi(t_a, t_b) = \begin{cases} \Delta V & \text{if } \Delta V > \Delta V_{max}^{lim} \\ -1/t_b & \text{otherwise} \end{cases} \quad (1)$$

- If the transfer at times $\{t_a, t_b\}$ is infeasible, the positive ΔV value is returned and can further be minimized until $\Delta V \leq \Delta V_{max}^{lim}$ is reached.

Computing Transfers



- We begin by developing a tool $opt\Delta V$ that finds the earliest transfer from one object to another one within a given time window $[t_s, t_d]$, given a ΔV -limit ΔV_{max}^{lim} .
- If no such transfer exists, the function returns an error.
- We approach this as numerical minimization problem with objective function ψ :

$$\psi(t_a, t_b) = \begin{cases} \Delta V & \text{if } \Delta V > \Delta V_{max}^{lim} \\ -1/t_b & \text{otherwise} \end{cases} \quad (1)$$

- If the transfer at times $\{t_a, t_b\}$ is infeasible, the positive ΔV value is returned and can further be minimized until $\Delta V \leq \Delta V_{max}^{lim}$ is reached.
- Once we found such transfer, $-1/t_b$ minimizes the arrival time t_b .

Computing Transfers



- We begin by developing a tool $opt\Delta V$ that finds the earliest transfer from one object to another one within a given time window $[t_s, t_d]$, given a ΔV -limit ΔV_{max}^{lim} .
- If no such transfer exists, the function returns an error.
- We approach this as numerical minimization problem with objective function ψ :

$$\psi(t_a, t_b) = \begin{cases} \Delta V & \text{if } \Delta V > \Delta V_{max}^{lim} \\ -1/t_b & \text{otherwise} \end{cases} \quad (1)$$

- If the transfer at times $\{t_a, t_b\}$ is infeasible, the positive ΔV value is returned and can further be minimized until $\Delta V \leq \Delta V_{max}^{lim}$ is reached.
- Once we found such transfer, $-1/t_b$ minimizes the arrival time t_b .
- We use a variety of algorithms, including Nelder&Mead, BFGS, and CMA-ES.

Computing Transfers



- We begin by developing a tool $opt\Delta V$ that finds the earliest transfer from one object to another one within a given time window $[t_s, t_d]$, given a ΔV -limit ΔV_{max}^{lim} .
- If no such transfer exists, the function returns an error.
- We approach this as numerical minimization problem with objective function ψ :

$$\psi(t_a, t_b) = \begin{cases} \Delta V & \text{if } \Delta V > \Delta V_{max}^{lim} \\ -1/t_b & \text{otherwise} \end{cases} \quad (1)$$

- If the transfer at times $\{t_a, t_b\}$ is infeasible, the positive ΔV value is returned and can further be minimized until $\Delta V \leq \Delta V_{max}^{lim}$ is reached.
- Once we found such transfer, $-1/t_b$ minimizes the arrival time t_b .
- We use a variety of algorithms, including Nelder&Mead, BFGS, and CMA-ES.
- If a feasible transfer with end time t' is found, we search again in $[t_s, t')$ until the search fails, retaining and returning the best result.

Infeasible Transfers and Preprocessing



- We first assemble a transfer matrix $\mathcal{T}$ for each instance, telling us whether we can travel from object to another either *not at all*, with excess velocity change, or with a normal velocity change.

Infeasible Transfers and Preprocessing



- We first assemble a transfer matrix $\mathcal{T}$ for each instance, telling us whether we can travel from object to another either *not at all*, with excess velocity change, or with a normal velocity change.
- The matrices $\mathcal{T}$ are sparse, most transfers are infeasible!

Infeasible Transfers and Preprocessing



- We first assemble a transfer matrix $\mathcal{T}$ for each instance, telling us whether we can travel from object to another either *not at all*, with excess velocity change, or with a normal velocity change.
- The matrices $\mathcal{T}$ are sparse, most transfers are infeasible!
- On instance **small**, only 2.8 other objects can be reached with $\Delta V \leq \Delta V_{max}$ in average!

Infeasible Transfers and Preprocessing



- We first assemble a transfer matrix $\mathcal{T}$ for each instance, telling us whether we can travel from object to another either *not at all*, with excess velocity change, or with a normal velocity change.
- The matrices $\mathcal{T}$ are sparse, most transfers are infeasible!
- On instance **small**, only 2.8 other objects can be reached with $\Delta V \leq \Delta V_{max}$ in average!
- The vast majority of visit sequences will not be feasible, so always using *opt* ΔV would be a huge waste of runtime. . .

Permutation Encoding



- We use an encoding based on permutations x of the first n natural numbers.

Permutation Encoding



- We use an encoding based on permutations x of the first n natural numbers.
- A permutation x is decoded to a solution vector y with the actual travel times using a three-stage decoding function γ .

Permutation Encoding: Stage 1



- First, $\gamma(x)$ processes x from beginning to end and divides it into *fragments* π using the transfer matrix $\mathcal{T}$.

Permutation Encoding: Stage 1



- First, $\gamma(x)$ processes x from beginning to end and divides it into *fragments* π using the transfer matrix $\mathcal{T}$.
- A fragment π is a sub-sequence of x such that normal ΔV can be used to travel from one object to the next.

Permutation Encoding: Stage 1



- First, $\gamma(x)$ processes x from beginning to end and divides it into *fragments* π using the transfer matrix $\mathcal{T}$.
- A fragment π is a sub-sequence of x such that normal ΔV can be used to travel from one object to the next.
- Fragments that can be stitched together by pre-pending and appending without violating this condition are merged.

Permutation Encoding: Stage 1



- First, $\gamma(x)$ processes x from beginning to end and divides it into *fragments* π using the transfer matrix $\mathcal{T}$.
- A fragment π is a sub-sequence of x such that normal ΔV can be used to travel from one object to the next.
- Fragments that can be stitched together by pre-pending and appending without violating this condition are merged.
- Fragments that can be inserted into each other without violating the condition are merged.

Permutation Encoding: Stage 1



- First, $\gamma(x)$ processes x from beginning to end and divides it into *fragments* π using the transfer matrix $\mathcal{T}$.
- A fragment π is a sub-sequence of x such that normal ΔV can be used to travel from one object to the next.
- Fragments that can be stitched together by pre-pending and appending without violating this condition are merged.
- Fragments that can be inserted into each other without violating the condition are merged.
- This is repeated with the relaxed requirement to allow excess velocity changes.

Permutation Encoding: Stage 1



- First, $\gamma(x)$ processes x from beginning to end and divides it into *fragments* π using the transfer matrix $\mathcal{T}$.
- A fragment π is a sub-sequence of x such that normal ΔV can be used to travel from one object to the next.
- Fragments that can be stitched together by pre-pending and appending without violating this condition are merged.
- Fragments that can be inserted into each other without violating the condition are merged.
- This is repeated with the relaxed requirement to allow excess velocity changes.
- Finally, all remaining fragments are stitched together regardless, yielding a new permutation x' .

Permutation Encoding: Stage 2



- We use the transfer matrix $\mathcal{T}$ to count the expected constraint violations in x' .

Permutation Encoding: Stage 2



- We use the transfer matrix $\mathcal{T}$ to count the expected constraint violations in x' .
- If constraint violations are expected, the decoding stops — this is very fast, no physics computation needed at all.

Permutation Encoding: Stage 3



- Otherwise, we process the permutation x' from beginning to end computing transfers using the actual physics computation $opt\Delta V$.

Permutation Encoding: Stage 3



- Otherwise, we process the permutation x' from beginning to end computing transfers using the actual physics computation $opt\Delta V$.
- We make sure that each solution uses all 5 permitted excess velocity changes (even if it does not need them), because then we can make travel faster.

Optimization



- We apply a randomized local search (RLS) to minimize the objective function f :

$$f(y) = \begin{cases} p(y) & \text{if } p(y) > 0 \\ -1/t_{tot}(y) & \text{otherwise} \end{cases} \quad (2)$$

Optimization



- We apply a randomized local search (RLS) to minimize the objective function f :

$$f(y) = \begin{cases} p(y) & \text{if } p(y) > 0 \\ -1/t_{tot}(y) & \text{otherwise} \end{cases} \quad (2)$$

- A solution y is infeasible if $p(y) > 0$, in which case the penalty p is returned.

Optimization



- We apply a randomized local search (RLS) to minimize the objective function f :

$$f(y) = \begin{cases} p(y) & \text{if } p(y) > 0 \\ -1/t_{tot}(y) & \text{otherwise} \end{cases} \quad (2)$$

- A solution y is infeasible if $p(y) > 0$, in which case the penalty p is returned.
- For feasible solutions, the total time t_{tot} is minimized.

Summary



- We tested several different algorithm setups implemented in moptipy⁷⁴.

Summary



- We tested several different algorithm setups implemented in moptipy⁷⁴.
- Our results are obtained with low computational cost on off-the-shelf laptops.

Summary



- We tested several different algorithm setups implemented in moptipy⁷⁴.
- Our results are obtained with low computational cost on off-the-shelf laptops.
- An early setup found a feasible solution with $t_{tot} = 2689.4$ for the **large** instance within 187 seconds.



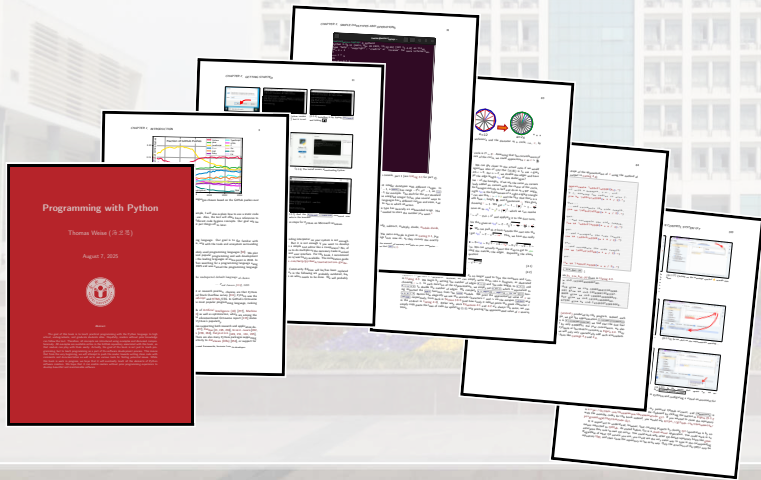
Advertisement



Programming with Python



We have a freely available course book on *Programming with Python* at <https://thomasweise.github.io/programmingWithPython>, with focus on practical software development using the Python ecosystem of tools⁷⁰.



Databases



We have a freely available course book on *Databases* at <https://thomasweise.github.io/databases>, with actual practical examples using a real database management system (DBMS)⁶⁷.



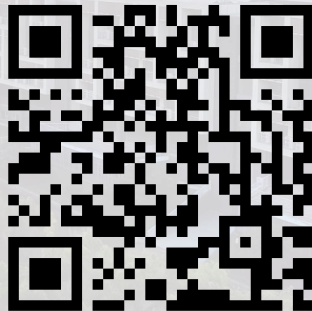
Metaheuristic Optimization in Python: moptipy



We offer moptipy⁷⁴, a mature open source Python package for metaheuristic optimization, which implements several algorithms, can run self-documenting experiments in parallel and in a distributed fashion, and offers statistical evaluation tools.

The collage consists of several overlapping screenshots from the moptipy project:

- Top Left:** GitHub repository page for `moptipy` by `thomaspeter`. It shows the README with a description of the package and a list of features.
- Top Right:** A section of the documentation titled "Installation", providing instructions on how to install the package using pip or from source.
- Middle Left:** The "Table of Contents" of the documentation, listing sections such as "Introduction", "How to Use", "How to Solve an Optimization Problem", and "Implemented Algorithms".
- Middle Right:** A code snippet showing how to run an experiment using the `moptipy` library, including setting up the problem, creating an instance, and running the optimization.
- Bottom Left:** A progress plot showing the results of an optimization process. The x-axis represents the number of iterations (from 0 to 200), and the y-axis represents the fitness value (from 0.0 to 10.0). The plot shows several curves representing different runs, with some reaching a plateau and others continuing to improve.

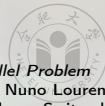




谢谢你们！
Thank you!
Vielen Dank!



References I



- [1] Denis Antipov and Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuño Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 117–128. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_10 (cit. on p. 64).
- [2] David Lee Applegate, Robert E. Bixby, Vašek Chvátal, and William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2nd ed. Vol. 17 of Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8. This book also discusses the Concorde Traveling Salesperson Problem (TSP) solver. (Cit. on pp. 61, 66).
- [3] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 978-1-4181-6963-3. URL: <http://gallica.bnf.fr/ark:/12148/bpt6k994750> (visited on 2023-12-13) (cit. on p. 67).
- [4] Thomas Bäck, David B. Fogel, and Zbigniew "Zbyszek" Michalewicz, eds. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd and Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (cit. on p. 61).
- [5] Tim Berners-Lee. *Re: Qualifiers on Hypertext links*. . . Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: <https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt> (visited on 2025-02-05) (cit. on p. 67).
- [6] Tim Berners-Lee, Larry Masinter, and Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 1994. URL: <https://www.ietf.org/rfc/rfc1738.txt> (visited on 2025-02-05) (cit. on p. 66).
- [7] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, May 2008. ISBN: 978-0-321-35668-0 (cit. on p. 62).
- [8] Rainer E. Burkard, Eranda Çela, Panos Miliotakis Pardalos, and Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miliotakis Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (cit. on p. 64).

References II



- [9] Eduardo Carvalho Pinto and Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell University Library, Dec. 3, 2018. doi:10.48550/arXiv.1812.00493. URL: <https://arxiv.org/abs/1812.00493> (visited on 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (cit. on p. 61).
- [10] Vladimír Černý. "Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm". *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 0022-3239. doi:10.1007/BF00940812 (cit. on p. 65).
- [11] Bo Chen, Chris N. Potts, and Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miliotopoulos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. (Cit. on p. 67).
- [12] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson, and Thomas Weise (汤卫恩). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (cit. on pp. 62–64).
- [13] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. May 3–5, 1971, Shaker Heights, OH, USA. Ed. by Michael A. Harrison, Ranajit B. Banerji, and Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, pp. 151–158. ISBN: 978-1-4503-7464-4. doi:10.1145/800157.805047 (cit. on pp. 63, 67).
- [14] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: <https://math.uwaterloo.ca/tsp/concorde> (visited on 2026-09-26) (cit. on p. 61).
- [15] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–Nov. 9, 2014. ISBN: 978-0-691-16352-9 (cit. on p. 61).

References III



- [16] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: 978-0-13-299045-5 (cit. on p. 67).
- [17] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0224-9 (cit. on p. 61).
- [18] Benjamin Doerr, Carola Doerr, and Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2014.11.028 (cit. on p. 64).
- [19] Stefan Droste, Thomas Jansen, and Ingo Wegener. "On the Analysis of the $(1 + 1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/S0304-3975(01)00182-7 (cit. on pp. 61–63, 65).
- [20] Stefan Droste, Thomas Jansen, and Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, July 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1007/S00224-004-1177-Z (cit. on p. 64).
- [21] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-564-1 (cit. on p. 64).
- [22] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt, and Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Win. 2009. Cambridge, MA, USA: MIT Press. ISSN: 1063-6560. doi:10.1162/evco.2009.17.4.17401 (cit. on p. 66).
- [23] Tobias Friedrich, Francesco Quinzan, and Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyoto, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 293–300. ISBN: 978-1-4503-5618-3. doi:10.1145/3205455.3205515 (cit. on pp. 62, 66).
- [24] Michael R. Garey and David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: 978-0-7167-1045-5 (cit. on p. 66).

References IV



- [25] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sum. 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.1.3.190 (cit. on p. 66).
- [26] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Win. 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.2.1.4 (cit. on p. 66).
- [27] Fred Glover and Gary A. Kochenberger, eds. *Handbook of Metaheuristics*. Vol. 57 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: 0884-8289. ISBN: 978-0-306-48056-0. doi:10.1007/b101874 (cit. on p. 63).
- [28] Fred Glover, Éric D. Taillard, and Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, Mar. 1993. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/BF02078647. Special issue on Tabu Search (TS). (Cit. on p. 66).
- [29] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: <https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf> (visited on 2025-08-01) (cit. on p. 67).
- [30] Gregory Z. Gutin and Abraham P. Punnen, eds. *The Traveling Salesman Problem and its Variations*. Vol. 12 of Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, May 2002. ISSN: 1388-3011. doi:10.1007/b101971 (cit. on p. 66).
- [31] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: 978-1-55860-872-6. doi:10.1016/B978-1-55860-872-6.X5016-1 (cit. on pp. 63, 65).
- [32] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: 1863-7310. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on p. 64).
- [33] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on p. 59).

References V



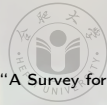
- [34] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. Oct. 5–7, 1982. Ed. by Joseph Allen Fisher, William J. Tracz, and Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and New York, NY, USA: Association for Computing Machinery (ACM), Oct. 1982, pp. 143–148. doi:[10.5555/800036.800944](https://doi.org/10.5555/800036.800944). See ³⁵ (cit. on p. 53).
- [35] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dec. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: [1050-916X](#). doi:[10.1145/1014194.800944](https://doi.org/10.1145/1014194.800944). See ³⁴ (cit. on pp. 53, 65).
- [36] Scott Kirkpatrick, C. Daniel Gelatt, Jr., and Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, May 13, 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: [0036-8075](#). doi:[10.1126/science.220.4598.671](https://doi.org/10.1126/science.220.4598.671). URL: <https://www.researchgate.net/publication/6026283> (visited on 2025-08-08) (cit. on p. 65).
- [37] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–June 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: [0163-5700](#). doi:[10.1145/1008328.1008329](https://doi.org/10.1145/1008328.1008329) (cit. on pp. 67, 68).
- [38] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: [978-0-201-89683-1](#) (cit. on pp. 67, 68).
- [39] Tjalling Charles Koopmans and Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: [0012-9682](#). doi:[10.2307/1907742](https://doi.org/10.2307/1907742) (cit. on p. 64).
- [40] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: [978-0-8218-2650-8](#) (cit. on p. 67).

References VI



- [41] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Ed. by Stephen C. Graves, Alexander Hendrik George Rinnooy Kan, and Paul H. Zipkin. Vol. IV of Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Chap. 9, pp. 445–522. ISSN: **0927-0507**. ISBN: **978-0-444-87472-6**. doi:[10.1016/S0927-0507\(05\)80189-6](https://doi.org/10.1016/S0927-0507(05)80189-6). URL: <http://alexandria.tue.nl/repository/books/339776.pdf> (visited on 2023-12-06) (cit. on p. 67).
- [42] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sept. 1985. ISSN: **0277-2698**. ISBN: **978-0-471-90413-7** (cit. on p. 66).
- [43] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: **1863-7310**. ISBN: **978-3-319-13071-2**. doi:[10.1007/978-3-319-13072-9](https://doi.org/10.1007/978-3-319-13072-9) (cit. on p. 64).
- [44] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson, and Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, July 2024. London, England, UK: Springer Nature Limited. ISSN: **1432-7643**. doi:[10.1007/s00500-024-09718-8](https://doi.org/10.1007/s00500-024-09718-8) (cit. on pp. 62, 63, 66).
- [45] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, and Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. Dec. 4–7, 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: **978-1-6654-8769-6**. doi:[10.1109/SSCI51031.2022.10022296](https://doi.org/10.1109/SSCI51031.2022.10022296) (cit. on pp. 62, 63).
- [46] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thüerer, Markus Wagner, and Thomas Weise (汤卫思). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 173–180. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0012888800003837](https://doi.org/10.5220/0012888800003837) (cit. on p. 63).

References VII



- [47] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn, and Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.09.032 (cit. on p. 64).
- [48] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: 978-1-4920-5627-0 (cit. on p. 62).
- [49] Mark Lutz. *Learning Python*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2025. ISBN: 978-1-0981-7130-8 (cit. on p. 64).
- [50] Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe, eds. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0000195000003837.
- [51] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 15–26. URL: <https://www.researchgate.net/publication/284805798> (visited on 2025-08-15) (cit. on p. 64).
- [52] Siegfried Nijssen and Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2002.806169 (cit. on p. 65).
- [53] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: 978-1-4919-6336-4 (cit. on p. 64).
- [54] Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham, eds. *Handbook of Combinatorial Optimization*. 1st ed. Boston, MA, USA: Springer, 1998. ISBN: 978-1-4613-7987-4.
- [55] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dec. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.18.6.1225 (cit. on p. 65).
- [56] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on p. 64).

References VIII



- [57] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/31EC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: <https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf> (visited on 2024-06-29) (cit. on p. 61).
- [58] Sanatan Rai and George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Ed. by Christodoulos A. Floudas and Panos Miliadis Pardalos. 2nd ed. Boston, MA, USA: Springer, Sept. 2008, pp. 2675–2682. ISBN: 978-0-387-74758-3. doi:10.1007/978-0-387-74759-0_462 (cit. on p. 67).
- [59] Soraya B. Rana and L. Darrell Whitley. "Genetic Algorithm Behavior in the MAXSAT Domain". In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. Sept. 27–30, 1998, Amsterdam, The Netherlands. Ed. by Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, pp. 785–794. ISSN: 0302-9743. ISBN: 978-3-540-65078-2. doi:10.1007/BFb0056920 (cit. on p. 63).
- [60] Gerhard Reinelt. "TSPLIB – A Traveling Salesman Problem Library". *ORSA Journal on Computing* 3(4):376–384, Aut. 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.4.376 (cit. on p. 66).
- [61] Gerhard Reinelt. *TSPLIB95*. Tech. rep. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–Apr. 28, 2016. URL: <https://github.com/shredderzsjw/TSPLIB> (visited on 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite> of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Cit. on p. 66).
- [62] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Vol. 35 of Forschungsergebnisse zur Informatik. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (cit. on p. 63).
- [63] Sartaj Sahni and Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, July 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: <https://sites.cs.ucsb.edu/~teo/papers/JACM-PCom.pdf> (visited on 2026-02-21) (cit. on p. 64).

References IX



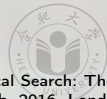
- [64] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: **978-1-80323-347-5** (cit. on p. **64**).
- [65] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇), and Thomas Weise (汤卫思). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Vol. 1. Sept. 14–18, 2024, Hagenberg, Mühlkreis, Austria. Ed. by Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck. Vol. 15148 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: **0302-9743**. ISBN: **978-3-031-70054-5**. doi:10.1007/978-3-031-70055-2_23 (cit. on pp. **62**, **63**).
- [66] Clarissa Van Hoyweghen, David Edward Goldberg, and Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. July 9–13, 2002, New York, NY, USA. Ed. by William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke, and Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, pp. 626–633. ISBN: **978-1-55860-878-8** (cit. on p. **66**).
- [67] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: <https://thomasweise.github.io/databases> (visited on 2025-01-05) (cit. on pp. **46**, **61**).
- [68] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: <https://www.researchgate.net/publication/200622167> (visited on 2025-07-25) (cit. on pp. **61**, **63**).
- [69] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: <https://thomasweise.github.io/programmingWithC> (visited on 2026-05-10) (cit. on p. **61**).
- [70] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: <https://thomasweise.github.io/programmingWithPython> (visited on 2025-01-05) (cit. on pp. **45**, **64**).

References X



- [71] Thomas Weise (汤卫思), Sina Abdipoor (思纳), Mingxuan Li (李明轩), and Zhize Wu (吴志泽). "Solving the SpOC 4 Challenge 2 with a Multi-Step Permutation Encoding for the Keplerian Traveling Salesperson". In: *28th Genetic and Evolutionary Computation Conference Companion (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026. ISBN: **979-8-4007-2488-6**. doi:[10.1145/3795101.3815557](https://doi.org/10.1145/3795101.3815557) (cit. on pp. 2–8).
- [72] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew "Zbyszek" Michalewicz, and Xin Yao (姚新). "Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem". *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: **1556-603X**. doi:[10.1109/MCI.2014.2326101](https://doi.org/10.1109/MCI.2014.2326101) (cit. on p. 66).
- [73] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert, and Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: **1089-778X**. doi:[10.1109/TEVC.2013.2251885](https://doi.org/10.1109/TEVC.2013.2251885) (cit. on p. 62).
- [74] Thomas Weise (汤卫思) and Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. July 15–19, 2023, Lisbon, Portugal. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: **979-8-4007-0120-7**. doi:[10.1145/3583133.3596306](https://doi.org/10.1145/3583133.3596306) (cit. on pp. 41–43, 47, 63).
- [75] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), and Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: **1089-778X**. doi:[10.1109/TEVC.2020.3032090](https://doi.org/10.1109/TEVC.2020.3032090) (cit. on pp. 61–66).
- [76] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩), and Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: **1089-778X**. doi:[10.1109/TEVC.2022.3191698](https://doi.org/10.1109/TEVC.2022.3191698) (cit. on pp. 61, 62, 65, 66).

References XI



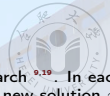
- [77] Thomas Weise (汤卫思), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂), and Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/s10898-016-0417-5 (cit. on p. 66).
- [78] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. June 1989, Fairfax, VA, USA: George Mason University. Ed. by J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, pp. 116–123. ISBN: 978-1-55860-066-9. URL: <https://www.researchgate.net/publication/2527551> (visited on 2025-08-08) (cit. on p. 63).
- [79] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg, and Thomas Weise (汤卫思). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 38–49. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012891500003837 (cit. on p. 63).
- [80] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: <https://www.techtarget.com/searchdatamanagement/definition/database-management-system> (visited on 2025-01-11) (cit. on p. 61).
- [81] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: <https://www.ietf.org/rfc/rfc3629.txt> (visited on 2025-02-05). See Unicode and ³³ (cit. on p. 66).
- [82] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽), and Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, pp. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098 (cit. on pp. 61–66).

References XII



- [83] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, and Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. July 14–18, 2024, Melbourne, VIC, Australia. Ed. by Xiaodong Li and Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (cit. on p. 63).
- [84] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, Tianyu Liang (梁天宇), Ming Tan (檀明), and Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (cit. on p. 63).

Glossary I



- (1 + 1) EA The (1 + 1) EA is a local search algorithm that retains the best solution x_c discovered so far during the search^{9,19}. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then the (1 + 1) EA uses a unary search operator that flips each bit independently with probability m/n , where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The (1 + 1) EA is a special case of the $(\mu + \lambda)$ evolutionary algorithm $((\mu + \lambda)$ EA) where $\mu = \lambda = 1$.
- (1 + 1) FEA The (1 + 1) evolutionary algorithm $((1 + 1)$ EA) with Frequency Fitness Assignment (FFA) plugged in^{75,76,82}. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where n is the dimension of the search space, distinguishing it from the unary operator in RLS with FFA (FRLS) which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.
- $(\mu + \lambda)$ EA The $(\mu + \lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu + \lambda$ solutions, from which then the best μ solutions are retained to form the parent population of the next generation. If the search space is the bit strings of length n , then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.
- C is a programming language, which is very successful in system programming situations^{17,57,69}.
- Concorde is an exact solver for the TSP^{2,14,15}, which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.
- DB A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases*⁶⁷.
- DBMS A *database management system* is the software layer located between the user or application and the database (DB). The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB⁸⁰.
- EA An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, respectively)^{4,68}.

Glossary II



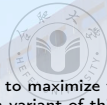
- ESA The *European Space Agency* is an international organization dedicated to space exploration, research, and technology development. Learn more at <https://www.esa.int>.
- FFA *Frequency Fitness Assignment* (中文: 频率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values^{73,75,76}. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the Maximum Satisfiability (MaxSAT)^{75,76}, the TSP^{44,45}, and the Quadratic Assignment Problem (QAP)^{12,65}, amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau^{75,76,82}. The most common realizations of FFA so far are the $(1 + 1)$ FEA and FRLS.
- GECCO The *Genetic and Evolutionary Computation Conference* is one of the largest conferences in the field of EAs. It has been established in 1999 and its proceedings are published by ACM. Learn more at <https://sig.sigevo.org/GECCOs>.
- Java is another very successful programming language, with roots in the C family of languages^{7,48}.
- Jump The Jump problem is a benchmark problem for bit-string based black-box search^{19,23,82}. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length n . However, the solutions representing the last ω bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the deceptive basin around the optimum that an algorithm needs to cross. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$ ¹⁹. In experiments, the $(1 + 1)$ EA with FFA ($(1 + 1)$ FEA) achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{75,76,82}.

Glossary III



- LeadingOnes** The LeadingOnes problem is a benchmark problem for bit-string based black-box search^{62,78,82}. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n \prod_{j=1}^i x_j$. Different from OneMax, the contribution to the objective value of the bit at index i thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$ ¹⁹. The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^3)$ ⁷⁵.
- MaxSAT** The goal of satisfiability problems is to find an assignment for n Boolean variables that make a given Boolean formula $F : \{0, 1\}^n \mapsto \{0, 1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT)³¹, F is given in conjunctive normal form, i.e., the variables appear either directly or negated in m "or" clauses, which are all combined into one "and." The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting x . If $f(x) = 0$, then all clauses of F are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$ -complete¹³ and exhibits low epistasis but deceptiveness⁵⁹, making it hard for local searches.
- metaheuristic** A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$ ^{27,68}. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions or similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are stochastic local searches (SLSes), Simulated Annealing (SA), TS, and EAs.
- moptipy** is the *Metaheuristic Optimization in Python* library⁷⁴. It has been used in several different research works, including^{12,44–46,65,79,83,84}. Algorithms implemented by moptipy include EAs, $(1+1)$ EA, RLS, SA, and the $(1+1)$ FEA, amongst others. Learn more at <https://thomasweise.github.io/moptipy> and <https://thomasweise.github.io/moptipyapps>.

Glossary IV



- OneMax** OneMax is the simplest possible benchmark problem for bit-string based black-box search^{20,51,82}. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of $(1 + 1)$ EA on OneMax is in $\Theta(n \ln n)$ ^{18,51}. The average runtime $(1 + 1)$ FEA on OneMax seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^2 \ln n)$ ⁷⁵.
- Plateau** The Plateau problem is a benchmark problem for bit-string based black-box search^{1,82}. It is structured exactly like Jump, but instead of being deceptive, the last ω bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and ω otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the neutral plane around the optimum that an algorithm needs to cross. An $(1 + 1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus¹. The average runtime of the $(1 + 1)$ FEA on Plateaus is slower by a factor seemingly proportional in n .
- PostgreSQL** An open source object-relational DBMS^{21,53,56,64}. See <https://postgresql.org> for more information.
- Python** The Python programming language^{32,43,49,70}, i.e., what you will learn about in our book⁷⁰. Learn more at <https://python.org>.
- QAP** The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations^{8,12,39,47}. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP, a distance matrix D is given, where D_{pq} specifies the distance from location p to location q , as well as a flow matrix F , where F_{ij} is the amount of material flowing from facility i to facility j . The objective function $f(x)$ then rates a permutation x as $f(x) = \sum_{i=1}^n \sum_{j=1}^n D_{x_i x_j} F_{ij}$. The QAP is $\mathcal{NP}$ -complete⁶³.

Glossary V



- RLS** Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.
- FRLS** The RLS with FFA plugged in ⁸². On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1 + 1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1 + 1)$ FEA.
- SA** Simulated Annealing is a SLS that sometimes accepts a worse solution ^{10,35,36,55}. The probability to do so decreases over time and with the difference in objective values, i.e., is the lower the worse the new solution is.
- SLS** *Stochastic local search or local search* for short, is one of the simplest metaheuristic search paradigms ³¹: A single “current” candidate solution is maintained and new solutions are sampled from its neighborhood. If these solutions are deemed interesting based on their objective value, they replace the “current”. The cycle continues. Basic examples are RLS and $(1 + 1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.
- SpOC** The *Space Optimization Competition* is a competition organized by the Advanced Concepts Team of ESA at GECCO since 2022, where various optimization problems related to space exploration and travel are posed and researchers worldwide can compete. Learn more at <https://optimise.esa.int>.
- Trap** The Trap problem is a benchmark problem for bit-string based black-box search ^{19,52,82}. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the “gradient” in the search space leads the search away from the global optimum. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = 0$ and $n - \|x\|_1 - 1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ¹⁹. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{75,76,82}.

Glossary VI



TS *Tabu Search* is one of the most successful SLSES^{25,26,28}. In order to avoid returning to previous solutions, it declares certain solutions or search moves as “tabu.” For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion).

TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are defined^{2,30,42,44,72,77}. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$ -hard combinatorial optimization problems^{24,30}.

TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP)^{60,61}. Its original URL was available until 2025 and then became defunct. The repository <https://github.com/shredderzwj/TSPLIB> contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite>, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of symmetric TSPs with scales n ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric TSPs with scales $n \in 17..443$.

TwoMax The TwoMax problem is a benchmark problem for bit-string based black-box search^{23,66,82}. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\{\|x\|_1, n - \|x\|_1\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$ ^{22,23}. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax^{75,76,82}.

URI A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet⁸¹. It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples for URIs.

URL A *Uniform Resource Locator* identifies a resource in the World Wide Web (WWW) and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites⁶. URLs are subsets of Uniform Resource Identifiers (URIs).

Glossary VII



WWW World Wide Web ^{5,16}

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j . For example, $5..9$ is equivalent to $\{5, 6, 7, 8, 9\}$

$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution y in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

$\gamma(x)$ The decoding function $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ translates point from the search space $\mathbb{X}$ to one point y in the solution space $\mathbb{Y}$. It is a very common case that $\mathbb{X} = \mathbb{Y}$ and that the decoding function γ is the identity mapping.

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$.

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer) ²⁹.

$\mathcal{NP}$ -complete A decision problem is $\mathcal{NP}$ -complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time ^{29,58}. A problem is $\mathcal{NP}$ -complete if it is $\mathcal{NP}$ -hard and if it is in $\mathcal{NP}$.

$\mathcal{NP}$ -hard Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$ -hard problems ^{11,13,41} have a worst-case runtime that grows exponentially with the problem scale. A problem is $\mathcal{NP}$ -hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time ²⁹.

$\Omega(g(x))$ If $f(x) = \Omega(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0 \forall x \geq x_0$ ^{37,38}. In other words, $\Omega(g(x))$ describes a lower bound for function growth.

$\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x) \forall x \geq x_0$ ^{3,37,38,40}. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.

Glossary VIII



$\Theta(g(x))$ If $f(x) = \Theta(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \Omega(g(x))$ ^{37,38}. In other words, $\Theta(g(x))$ describes an exact order of function growth.

$\mathbb{R}$ the set of the real numbers.

$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

$\mathbb{X}$ The search space $\mathbb{X}$ is the space through which an optimization algorithm navigates in order to find good solutions. The search operators of metaheuristics are used to sample points $x \in \mathbb{X}$, for example. On many benchmark problems, the objective functions f directly evaluate the elements $x \in \mathbb{X}$, i.e., are defined as $f : \mathbb{X} \mapsto \mathbb{R}$. However, very often these elements x are not the same data structures that the user is interested in and/or that the objective functions evaluate: It is common that the search space $\mathbb{X}$ is a simple and plain data structure that can be tackled with standard operations or mathematical tools, such as real vectors $\mathbb{R}^n$, permutations, or bit strings, whereas the solution space $\mathbb{Y}$ can be something more complicated, like a Gantt chart. Then, the user only cares about the solutions $y \in \mathbb{Y}$ and not about the search space $\mathbb{X}$. A decoding $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ is employed to translate the $x \in \mathbb{X}$ to $y = \gamma(x)$ with $y \in \mathbb{Y}$.

x a point in the search space $\mathbb{X}$.

$\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions y . These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.

y a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.

$\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$, and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.